

SwegHammer

Project Handbook & Living To-Do List

Eddie Handford & Jake

Last updated: July 8, 2026

Contents

1	How to use this document	1
2	Project vision	1
3	The four goals	2
4	Where the project stands today	3
4.1	What works	3
4.2	What's missing in one sentence	3
5	Sprint plan: May 2026 handover (Ed/Jake)	3
6	Game phases	4
6.1	Command phase	4
6.2	Movement phase	5
6.3	Shooting phase	5
6.4	Charge phase	6
6.5	Fight (melee) phase	6
6.6	Morale / battleshock phase	6
7	Terrain and map	7
7.1	Coordinate system	7
7.2	Terrain types	7
7.3	Line of sight	7
8	Visualization: the dual-mode plan	7
9	Front end	8
10	AI Lab — evolved strategy sandbox (outside the two-stage pipeline)	8
11	Datasheet ingestion	9
12	Combat math reference card	10
12.1	Hit / wound / save chain	10
12.2	Lanchester score	10
12.3	Charge probability	10
12.4	Primary VP scoring and win condition	10
12.5	Strategy layer	11

13 Two-track points calibration	11
13.1 Track 1 — Sweg-balancer (Monte Carlo bisection)	11
13.2 Track 2 — Equilibrium (closed-form log-LSQ)	11
13.3 Why two tracks	13
14 Rule-citation system	13
14.1 Why	13
14.2 Format	13
14.3 Enforcement	14
15 Matchup-matrix calibration (evaluate_vs_meta)	14
15.1 How it runs	14
15.2 Reference data	14
15.3 Current state	14
16 Collaboration workflow	15
16.1 Ownership at a glance	15
16.2 Picking the project back up	15
16.3 When something is safe to merge to main	15
17 Glossary	16

1 How to use this document

This is the single living reference for SwegHammer. Open it when you sit down, close it when you walk away. It contains the project vision, the math behind each combat phase, a checklist of everything left to do, and ownership tags so we both know who is on what.

For Claude sessions: the operational rules (test-before-push, doc-update workflow, branch naming, git identity, catalogue layering) live in `CLAUDE.md` at the repo root. Read that file as well — it is auto-loaded into the Claude context for every session.

Conventions

- open task ★ in progress ✓ done
- [E] Eddie's slice [J] Jake's slice [?] unclaimed

When you pick a task up, swap its tag to your initial and flip the marker to ★. When you finish, flip to ✓ and commit the `.tex` change alongside the code change so the doc never drifts from reality.

Build

```
pdflatex OVERVIEW.tex
pdflatex PROJECT.tex && pdflatex PROJECT.tex
```

Build `OVERVIEW.pdf` first because `PROJECT.tex` embeds it as the pipeline figure. Then two passes on `PROJECT.tex` so the table of contents resolves.

2 Project vision

SwegHammer is a data-driven Warhammer 40K simulator with two goals:

1. Derive fair, transparent points costs from Lanchester's Square Law plus empirical simulation.

2. Test minimal rule tweaks (unit-by-unit activation, command-point compensation for the smaller army) that fix structural feel-bad moments like first-player advantage.

The simulator is the experimental apparatus. It must be *fast* enough to run thousands of battles in a calibration sweep, and *detailed* enough to render a single battle as a watchable replay for sanity-checking the model.

Pipeline structure. The project runs as two sequenced feedback loops, not one. Stage 1 (Goal A below) tunes the simulator’s rules until per-faction win rates match the tournament aggregate; Stage 2 (Goal C below) freezes the rules and fits a single master points equation that prices every unit from its stats (plus small per-unit residuals for outliers), tuning the equation’s stat coefficients and residuals until win rates across the catalogue flatten. The loop’s output is the equation, not a hand-tuned price list. The figure below shows the relationship; `OVERVIEW.tex` is the standalone shareable version with non-technical wording.

3 The four goals

As of the 2026-05 reorganisation, project work is tracked against four goals rather than the previous phase numbering. The phases that delivered the substrate (Phase 0 / 1 / 1.5 / 1.6, Phase A2 / B / C / D / E) are shipped and live in `ROADMAP.md` as a historical record. Active work tracks against the goals below.

Goal A — Sim matches real tournament data

Drive the per-faction matchup-matrix MAE down by implementing the faction-specific mechanics that distinguish factions in real play. Headline metric: noise-gated MAE vs the May 2026 Warp Friends tournament aggregate (per-faction errors counted only beyond each faction’s noise floor), currently gated 6.70 pts / raw 10.05 pts at $N=80$, target gated $\rightarrow 0$. As of 2026-05-15, 9 of 10 faction army rules ship (Tyranids Synapse, Drukhari Pain Tokens, AdMech Doctrina, GSC Cult Ambush, World Eaters Blood Tithe, Death Guard Disgustingly Resilient + Contagions, Thousand Sons All Is Dust, Necrons Reanimation, Orks WAAAGH! + Mob Rule). Marines Oath of Moment remains [NEEDS RETRY]. As of 2026-06-12 Astra Militarum Voice of Command (Orders) also ships: real REGIMENT / SQUADRON keyword targeting, a First Rank Fire rapid-fire gate, and squad-level Order dispatch. Plus 5 core 10e rules shipped this session: Look Out Sir, Lone Operative, Deadly Demise X, Fall Back, Desperate Escape. See `scripts/evaluate_vs_meta.py`.

Goal B — Equal-quality simulation per faction

Bring every faction to the same depth of in-game representation. As of 2026-05-15: 25 detachments registered (up from 21), 5 new per-faction stratagems on top of the 4 universal core + 3 original detachment sets, 5 Warlord enhancements, 4 CP-discount HQs, 32 leader abilities all with verbatim Wahapedia citations, 122/122 rules cited. A mechanic-fire validator (`tests/test_faction_mechanic_smoke.py`) confirms 19 implemented faction mechanics actually trigger in seeded battles.

Goal C — Fit the SwegHammer points equation (two-track)

Replace GW’s commercially-driven, hand-tuned price list with one master points equation: a closed-form formula mapping a unit’s stats to its cost, plus small per-unit residuals for outliers the formula cannot explain. Two independent solvers feed the fit in parallel; the divergence between them is itself a calibration signal:

- **Sweg-balancer** (`code/balancer.py`) — Monte Carlo bisection. For each unit, bisect points-per-model until armies-of-U vs armies-of-baseline-peer land at $50\% \pm 5\%$. Slow, but exercises *every* rule in the simulator (movement, range, terrain, charge variance, objective contest, CP).

- **Equilibrium** (`code/equilibrium.py`, Ed’s solver) — closed-form Bradley-Terry / log-LSQ on the pairwise time-to-kill matrix. All 6 phases shipped: 1 (shoot), 2 (+melee), 3 (defensive audit), 4 (tactical utility), 5 (meta-weighted), 6 (Nash mixed- strategy).

A cross-validation script (`scripts/cross_validate_pricing.py`) compares the two signals; 2 disagreements (Wraithguard, Crisis Sunforge) and 5 confirmations on the live catalogue. Both tracks are blocked on Ed’s points-per-model import fix — until `points_cost` returns the GW per-model cost rather than the Lanchester-derived score, the sim’s army budgets run in wrong-currency.

Goal D — Price non-damaging abilities

Speed, deep strike, scout, sticky objective, OC bonuses, re-deploy — abilities that don’t show up in the pairwise damage matrix but are real points value. Implemented as Equilibrium Phase 4’s `tactical_value(u)` multiplicative log-overlay on Phase 2. Weights grid-search calibrated on 9 anchors; the anchor set under- identifies `w_move`, `w_oc`, `w_infiltrator`, `w_deep_strike` (all calibrate to 0). Widening the anchor set is the next Goal D step.

4 Where the project stands today

4.1 What works

- ✓ Stochastic combat engine (`code/simulator.py`): roll-to-hit, AP-adjusted armour save, cover modifier, damage.
- ✓ Unit catalogue built from BSData WH40k 10e (v10.6.0) via `code/bsdata/`, with ~1294 units across all major factions. Override layer in `data/overrides.json`; calibrated point costs cached in `data/calibrated_points.json`.
- ✓ Unit-by-unit alternating activation (`Battle._run_round`).
- ✓ Command-point bonus for the smaller army from Round 2.
- ✓ Streamlit dashboard (`app.py`) with preset battles, attrition curves, survivor histograms, points-vs-winrate sweep.

4.2 What’s missing in one sentence

The current “combat” is a single abstract attack action. There are no *phases* (movement / shooting / charge / melee / morale), no map, no positions, and no terrain beyond a binary cover flag. The rest of this document is the plan to fill that in.

5 Sprint plan: May 2026 handover (Ed/Jake)

This section captures the architecture clarification and near-term work sequence agreed between Ed and Jake on 2026-05-15. It supersedes any ad-hoc next-steps notes scattered through the older sections.

Architecture clarification

SwegHammer is two interlocking systems, not one:

- **Equilibrium model** (`code/equilibrium.py`) — the core mathematics-hammer engine. Lancaster-derived, multi-dimensional, non-linear. Computes pairwise unit matchup outcomes in closed form. *This is what ships in the final product.*

- **Simulator** (`code/simulator.py`) — exists to tune the utility factors (deep strike, sticky objectives, scout, re-deploy and so on) that feed back into the equilibrium model. Slow and iterative; only needed for calibration.

Why the split matters. The end-user-facing front end (“Fire Prism vs Wraithknight — what are the odds?”) runs the equilibrium equation only. The simulator stays in the toolbox for the specific case where Games Workshop releases a weird new model and we need to derive its tactical utility weights before plugging it into the equation.

Calibration constraint. Real tournament data is scarce. Even a major event like the Las Vegas Open produces only a few thousand games across roughly twenty factions — too thin for faction-vs-faction matchup matrices on its own. The simulator covers the gap by generating the synthetic matchup volume the analytic solver needs.

Current focus (Jake)

Drive the simulator’s mean absolute error down to 2–4 % against real tournament results, using the Games Workshop printed point costs as the baseline for army budgets. The current honest reading is noise-gated 6.70 points at $N=80$ (see §3 Goal A); 2–4 % is the threshold at which the simulator’s outputs are trustworthy enough to drive points re-derivation in production.

Near-term sequence (priority order)

1. ○ [E] Speed up the user interface for faster iteration. The convergence tab is the right shape; extend the same live-streaming pattern to the calibration sweep so a tuning loop does not block on a multi-minute wait. See also the longer-term front-end items in §9.
2. ★ [J] Get the simulator’s mean absolute error down to 2–4 % against the May 2026 Warp Friends aggregate. Headline metric for this sprint; everything downstream blocks on it.
3. ○ [?] Generate a trial balanced points dataset. Once the mean absolute error is in the 2–4 % band, run the two-track points solvers (`code/balancer.py` and `code/equilibrium.py`, see §3 Goal C) and cache the output as a candidate full-catalogue re-pricing.
4. ○ [?] Sanity-check the trial dataset, then play-test in person. After the cross-validation script (`scripts/cross_validate_pricing.py`) clears the obvious outliers, approach local game groups for a structured blind play-test round — unit costs swapped in without telling players which units have been re-priced, then a post-game survey on perceived fairness.

Final product vision. When this sprint lands, the shipped product is the front end running the equilibrium equation on the trial-balanced points dataset, with the simulator preserved as a calibration utility for future Games Workshop releases.

6 Game phases

40K splits a turn into discrete phases. We will mirror that structure but keep each phase pluggable so we can disable expensive ones for fast calibration runs.

6.1 Command phase

At the start of a unit’s activation: gain 1 CP, resolve start-of-turn abilities, apply the smaller-army CP compensation rule.

To do

- [E] Add a `CommandPhase` step run at activation start.
- [E] Move existing CP bonus logic out of `Battle._run_round` into the new phase so it lives in one place.

6.2 Movement phase

A unit moves up to its Move characteristic in inches. Optionally Advance (+d6, no shooting that turn) or Fall Back (no shooting, no charging).

Math. Position update for a unit at $\mathbf{p}_t$ choosing destination $\mathbf{p}_{t+1}$ subject to

$$\|\mathbf{p}_{t+1} - \mathbf{p}_t\| \leq M_{\text{unit}} \cdot \sigma_{\text{terrain}},$$

where $\sigma_{\text{terrain}} \in (0, 1]$ is the terrain-slowdown factor on the line the unit traverses.

To do

- ✓ [E] Add `position: (float, float)` to `Unit`.
- ✓ [E] Add `move: float (inches)` to `UnitProfile`.
- ✓ [E] Implement `MovementPhase` – move-then-shoot sub-phase pair per activation. Targeting is now objective-aware via `code/strategy.py` (HOLD / CAPTURE / STEAL / ENGAGE / REPOSITION intents); units sitting on objectives stay put rather than abandoning VP to chase the nearest enemy.
- ✓ [?] Advance roll (+d6, no shooting that turn) implemented; range threshold is weapon range when engaging and objective control radius when capturing. Fall Back still unimplemented.
- [?] Segment-vs-impassable-terrain collision (only the destination is currently checked, so units can tunnel through walls).
- [?] Screen the held kiting lever (`SWEG_KITING` – shooters step back from melee-superior threats to maintain the stand-off gap) at eighty battles per matchup against a fresh anchor; adopt default-on if the read is a wash, keep it held if decisively worse. Its companion, the charge-trade veto (`SWEG_CHARGE_TRADE`), is already adopted default-on by owner ruling (2026-07-07).

6.3 Shooting phase

For every weapon in range and with line of sight: roll to hit, roll to wound, target rolls armour save, apply damage. **The current sim skips the wound roll**, which is the biggest single bit of math debt.

Math. Expected damage of one shot:

$$\mathbb{E}[D] = P_{\text{hit}} \cdot P_w(S, T) \cdot (1 - P_{\text{save}}(\text{sv}, \text{AP}, \text{cover})) \cdot d,$$

where the wound probability P_w follows the strength-vs-toughness table:

$$P_w(S, T) = \begin{cases} 5/6 & S \geq 2T \\ 4/6 & S > T \\ 3/6 & S = T \\ 2/6 & S < T \\ 1/6 & 2S \leq T \end{cases}$$

and the save probability uses an effective save characteristic $sv^* = sv - AP - \text{cover}$ capped at 2:

$$P_{\text{save}}(sv^*) = \max\left(0, \frac{7-sv^*}{6}\right).$$

To do

- ✓[E] Add `strength: int` to `UnitProfile`.
- ✓[E] Add `toughness: int` to `UnitProfile`.
- ✓[E] Implement the wound table above as `wound_probability(S, T)` and wire it into `Unit.attack()` between the hit and save rolls.
- [E] Regenerate `BASELINE.md` catalogue table to reflect the wound-roll-aware point costs (existing table is now stale; mirror match holds on symmetric terrain, but cross-faction matchups need recalibrating).
- ✓[E] Add `range_inches: int` to the weapon profile; gate shooting on range and LoS. Liang-Barsky segment-vs-rectangle intersection handles obscuring terrain.

6.4 Charge phase

Declare a charge against an enemy within 12". Roll $2d6$; if the total is at least the distance to the target, move into base contact.

Math.

$$P_{\text{charge}}(d) = \Pr[2d_6 \geq d] = \frac{\#\{(a, b) \in [1, 6]^2 : a + b \geq d\}}{36}.$$

Useful values: $P(6'') = 26/36 \approx 72\%$, $P(9'') = 10/36 \approx 28\%$, $P(12'') = 1/36 \approx 3\%$.

To do

- ✓[?] Implement `ChargePhase` with declaration, $2d6$ roll and move-into-engagement (1" of target). Lives in `Battle._do_charge`; gated by a charge-desire heuristic so pure shooters don't suicide-charge out of range.
- ✓[?] `_charging_this_round` set tracked on `Battle`; chargers fight first in the melee phase.

6.5 Fight (melee) phase

Units in base contact fight. Each model makes `attacks` attacks at `weapon_skill`, then wound, save, damage as per shooting.

To do

- ✓[?] `UnitProfile` now carries `melee_attacks`, `melee_damage_per_shot`, `melee_hit_probability`, `melee_strength`, `melee_ap`, `melee_weapon`. Mapper extracts `typeName="Melee Weapons"` profiles and picks a best-legal loadout; 1261 of 1291 units carry a usable melee profile.
- ✓[?] `MeleePhase` (`Battle._do_fight`): any unit within 1.5" of an enemy strikes with its melee profile; chargers fight first. `Unit.attack(..., mode="melee")` switches the stat block.

6.6 Morale / battleshock phase

A unit that lost models tests Leadership; a failed test inflicts further casualties or status effects.

To do

✓[?] `UnitProfile` carries `leadership` (Ld) and `oc` (Objective Control); both are extracted by the mapper from the `BSData` unit profile.

✓[?] 10e Battleshock implemented in the simulator: from Round 2, any unit below half HP rolls $2d6$ vs Ld; a fail leaves it Battleshocked for the round (OC drops to 0 for objective scoring). Detachment `enemy_ld_penalty` composes into the target.

7 Terrain and map

7.1 Coordinate system

Continuous 2D plane, units in inches. Standard boards: $44'' \times 60''$ (Combat Patrol), $44'' \times 90''$ (Strike Force). Models are circles with a defined base radius.

7.2 Terrain types

Type	Effect	Notes
Open	none	default
Light cover	+1 save vs ranged	matches today's "in cover" flag
Heavy cover	+1 save and -1 to hit	ruins, barricades
Obscuring	blocks LoS through it	forests, walls
Impassable	blocks movement	cliffs

7.3 Line of sight

Naive implementation: draw a segment from the attacker centre to the target centre, check intersection with any obscuring polygon. If that becomes a perf bottleneck, swap in a spatial hash over terrain polygons.

To do

✓[E] `Terrain` dataclass (axis-aligned rectangles for now; polygon upgrade can wait).

✓[E] `Map` dataclass: board size, terrain list, deployment-zone depth, and objective markers. Every stock map ships a 5-objective quincunx (centre + four equidistant markers $\sim 30\%$ in from each edge), matching the geometry of 10e Take and Hold / Crucible / Tipping Point. `Objective` dataclass in `code/map.py`: (`x`, `y`, `control_radius=3`", `vp_per_round=5`).

✓[E] `has_line_of_sight(attacker, target)` on `Map`, Liang-Barsky parametric clipping over obscuring rectangles. Units inside obscuring terrain can still shoot out and be shot at (the containing rectangle is excluded from the intersection test).

✓[E] `cover_at(point)` on `Map` returning the strongest cover the point sits inside; terrain-aware cover is applied per-shot in `Battle._do_shoot`.

✓[E] Three stock maps registered in `STOCK_MAPS`: `combat_patrol`, `open_plains`, `urban_sprawl`. The first two are point-symmetric reference maps; Urban Sprawl is deliberately asymmetric.

8 Visualization: the dual-mode plan

The simulator needs to do two things that pull in opposite directions:

1. Run thousands of fast battles for calibration sweeps.

2. Render a single battle in detail for sanity checks and demos.

Design. Keep the simulation core (`simulator.py`, `army.py`, `units.py`) *completely headless and deterministic given a seed*. Build rendering on top as an event-subscriber layer:

- **Detailed mode** – a *Pygame* or *matplotlib-animation* renderer with units drawn as tokens on a 2D top-down map, with range circles, LoS lines, charge arcs. Used for: replaying a single battle, demo videos, debugging movement bugs.
- **Fast mode** – *no rendering at all*, just numerical state. Used for: Monte Carlo sweeps where we run 10^3 – 10^4 battles per matchup and only care about win rates and attrition curves.

Crucially, the two modes share *the same* `Battle` object; the mode flag just toggles which subscriber is attached. A bug in the visualisation therefore cannot affect calibration numbers.

To do

- ✓ [E] Refactor `Battle.run()` to emit events via a subscriber list. Events defined in `code/events.py`: `BattleStarted`, `RoundStarted`, `UnitActivated`, `UnitMoved`, `UnitShot`, `UnitKilled`, `RoundEnded`, `BattleEnded`.
- ✓ [E] Null-subscriber fast mode – empty subscriber list means zero per-event overhead in calibration sweeps.
- [?] `PygameRenderer` – consumes the event stream, paints frames live.
- ✓ [E] Matplotlib static-frame renderer (`code/renderer.py`) reconstructing world state from the event stream; feeds the Streamlit replay page (a per-round, squad-level overview with one board snapshot per round). Renders advance / charge / melee / objective events with directional arrows, star bursts on hits, and gold rings around scoring objective markers.

9 Front end

The current Streamlit app (`app.py`) is a good v1 dashboard. We will extend it rather than replace it.

Near-term

- ✓ [E] Embed a scrub-through replay of one battle on a map (Streamlit "Watch a battle" tab; matplotlib top-down view with deployment-zone hints, terrain tiles, unit dots, move/shot arrows on the current event).
- ✓ [E] Map picker – sidebar dropdown over `STOCK_MAPS` (one map for now; structure ready for more).
- [E] Per-unit cover and terrain overrides in the sidebar (current cover flag is army-wide).

Longer-term

- [?] Army builder UI: drag-and-drop datasheets, points totalled live.
- [?] Calibration browser: view the most recent sweep, drill into outlier matchups.

10 AI Lab — evolved strategy sandbox (outside the two-stage pipeline)

An exploratory sandbox (2026-07): a genetic algorithm evolves five small, readable piloting knobs — charge aggression, engage-range buffer, melee-commitment threshold, hold-position bias, kiting stand-off — for one Intercessor Squad against a frozen baseline strain, over seeded five-model mirror duels. A generation’s champion that *clearly* beats the baseline (a fresh-seed confirmation batch whose Wilson 95 % lower bound clears 50 %) is promoted to *be* the new baseline, and the succession of promoted champions forms a strain lineage (`docs/ai_lab_lineage.csv`).

Deliberately **outside** both feedback loops of §3: it touches neither the tournament mean-absolute-error gate (Stage 1) nor any points file (Stage 2), and none of it models a Warhammer 40k rule — the genome rides pilot hooks (`Battle._pilot_move / _pilot_charge`, following the pre-existing `_pilot_focus` precedent), each structurally inert at its neutral value, so the neutral genome reproduces production behaviour byte-for-byte (proven event-for-event in `tests/test_ai_lab_pilot_hooks.py`).

Lives in `code/ai_lab/` with a headless runner (`scripts/ai_lab_run.py`) and a dashboard tab (“AI Lab”: live fitness chart, champion-versus-baseline replay, lineage table). Design document, gene table, and the deferred expansion genes (which enemy model to focus fire on — Ed’s flagged next knob — whom to charge, cover-seek radius) are catalogued in `docs/AI_LAB_PLAN.md`.

11 Datasheet ingestion

Importing real datasheets at scale is Jake’s slice of the project. The shape of the data model needs to be agreed before he starts, otherwise we will end up rewriting `units.py` twice.

Status: v1 complete (BSDData WH40k 10th-ed, ~1100 units).

- ✓ [J] Pick a source format — BSDData WH40k 10e BattleScribe XML files, pinned to release v10.6.0. See `code/bsdata/fetch.py` for the GitHub-backed cache layer.
- ✓ [J] Define a target schema — `MappedUnit` (`code/bsdata/mapper.py`) lowers to `UnitProfile` via the loader in `code/bsdata/loader.py`, merging `parsed.json` with user-tunable `data/overrides.json`.
- ✓ [J] Build a loader yielding `UnitProfile` instances — `UNIT_CATALOG` is now built at import time from the merged BSDData + overrides view.
- ✓ [J] Multi-weapon flattening — the mapper picks the single best-legal weapon by $A \cdot P_{\text{hit}} \cdot D \cdot (1 - P_{\text{save_baseline}})$, i.e. the loadout maximising expected damage through baseline-Marine armour. List of alternatives is kept in `MappedUnit.loadout` for audit.

Phase 2 follow-ups (see `ROADMAP.md`).

- ✓ [E] **Points-per-model cost is imported wrong.** The per-model points value coming through the BSDData → `UnitProfile` pipeline did not match the printed datasheet cost. The fix landed in three passes; the third closes the bug class.

2026-05-16 (commit c084ba0): `_squad_group_size` was reading the outer unit `selectionEntry`’s own `field="selections"` constraints as squad size. For squads with no inner `selectionEntryGroup` (Pink Horrors, Plaguebearers, Bloodletters, Sagittarum Custodians, Chaos Spawn, Obliterators, Kastelan Robots, &c.) the only such constraint is `scope="force"/"roster"/"parent"` — an army-list cap, not the model count — so `min_models` collapsed to 1 and per-model cost became the full squad cost (Pink Horrors: 140 pts/model instead of 14). Fix skipped entry-self constraints on unit entries and used shape (c) (sum per-model `scope="parent"` constraints). 73 of 156 suspicious multi-model squads were corrected.

2026-05-17 (branch *claudio/fixinfequiveexplorer*), *leader-plus-body shapes*: `extract_squad_size` stopped returning the first non-None group, which silently truncated leader-plus-body squads by one model (Devastator Squad read 30 pts/model instead of 24 — sergeant dropped). It now sums sizes across all outer groups and adds the unit entry's own direct `selectionEntry type="model"` children, so the leader-plus-direct-body shape (T'au Breacher Team: 1-model Shas'ui group + 9-model direct body = 10; Servitor Battleclade: 1 group + 3 direct = 9 models) reads correctly.

2026-05-17, *implicit-via-cost-tier shapes*: units whose squad size is encoded only in a points-tier modifier (`<modifier type="set" field=pts><condition type="greaterThan" field="selections" childId="model" value="N">`) now read N as the upper bound of the base cost tier. Jakhals (65 pts $\rightarrow$ 6.5 pts/model at $N=10$) and Neurogaunts (45 pts $\rightarrow$ 4.1 pts/model at $N=11$) were the canonical victims; cost-tier inference fires whenever the static signal under-reports.

2026-05-17, *stale balancer overrides*: the `points_override` entries written by `sweg_balance_mc` for Crisis Fireknife, Crisis Sunforge, Deathshroud Terminators, Plague Marines, Rubric Marines, Scarab Occult Terminators, and Hearthkyn Warriors had been calibrated against the pre-fix `min_models` (so the per-model value was implicitly the per-*squad* cost). With the model counts corrected, these inflated squad costs to 2.5–3 $\times$ GW MFM and priced Crisis suits out of the T'au Kauyon archetype seed budget. Stale entries removed from `overrides.json`; future balancer runs will repopulate them against the now- correct model counts.

✓ [?] **Regression test for the `extract_squad_size` bug class.** `tests/test_mapper.py::SquadSizeShapeRe` pins one canonical victim per encoding shape (Intercessor Squad, Devastator Squad, Plaguebearers, Breacher Team, Servitor Battleclade, Pink Horrors, Neurogaunts, Jakhals) against its known correct (`min`, `max`), so silent regressions surface in CI rather than in the equilibrium graph weeks later.

○ [J] Extract S (weapon Strength) and T (unit Toughness) from `BSData` into `UnitProfile` — currently defaulted to $S=T=4$ via `BASELINE_*`.

○ [J] Fall-back to melee weapons for units with no ranged profile (~ 260 currently flagged `enabled: false`).

○ [J] Decide per-squad vs per-model aggregation for `health` (currently per-model wounds).

12 Combat math reference card

12.1 Hit / wound / save chain

For one attack with weapon profile $(P_{\text{hit}}, S, \text{AP}, d)$ against target (T, sv) and cover bonus $c \in \{0, 1\}$:

$$\mathbb{E}[\text{damage}] = P_{\text{hit}} \cdot P_w(S, T) \cdot (1 - P_{\text{save}}(\text{sv} - \text{AP} - c)) \cdot d.$$

12.2 Lanchester score

$$\text{Score}(\text{unit}) = (H \cdot D \cdot P_{\text{hit}})^2, \quad \text{Score}(\text{army}) = \sum_i e_i^2.$$

12.3 Charge probability

$$P_{\text{charge}}(d) = \frac{1}{36} \sum_{a=1}^6 \sum_{b=1}^6 \mathbf{1}[a + b \geq d].$$

12.4 Primary VP scoring and win condition

At end of each round, for every objective o on the map let OC_o^A, OC_o^B be the total Objective Control of each side’s alive units inside the objective’s control radius (Battleshocked units contribute $OC = 0$). The side with the strict majority banks $o.vp_per_round$ Primary VP that round.

$$VP^A += \sum_{o \in O} o.v \cdot \mathbf{1}[OC_o^A > OC_o^B], \quad VP^B \text{ symmetrically.}$$

Win condition (replaces “most surviving units”):

1. One-sided wipe: if only one side has alive units, that side wins.
2. Else: higher Primary VP wins.
3. VP tie: higher remaining army points wins (10% margin else draw).

12.5 Strategy layer

Movement targeting is no longer “march at the nearest enemy”. The strategy layer (`code/strategy.py`) selects a (`target_pos, intent`) per activation from five intents: HOLD, CAPTURE, STEAL, ENGAGE, REPOSITION. Objectives are scored by current control state (enemy-held $\times 3.5$ to bias steals, uncontested $\times 2.5$, ours $\times 1.0$) and distance-weighted; role bias (`code/roles.py`, classifying units into SHOOTY / MELEE / DUAL / HORDE / HEAVY / SUPPORT) tips ties towards holding when in range vs always-closing.

13 Two-track points calibration

Goal C runs two independent points solvers. They disagree on purpose; the disagreement is the calibration signal.

13.1 Track 1 — Sweg-balancer (Monte Carlo bisection)

`code/balancer.py`; output `data/calibrated_points.json`.

For each candidate unit U :

1. Build two equal-points armies: one of U , one of a role-matched baseline peer (SHOOTY $\rightarrow$ Intercessor, MELEE $\rightarrow$ Assault Intercessor, HORDE $\rightarrow$ Boyz, HEAVY $\rightarrow$ Knight, SUPPORT $\rightarrow$ Terminator Captain).
2. Simulate N full battles on `combat_patrol`, with detachments wired on both sides.
3. Bisect U ’s points-per-model until the empirical win rate lands inside $[45\%, 55\%]$.

Three modes:

- **Homogeneous** (default) — same-role peer.
- **-leader-attached** — for CHARACTER units, the candidate is built as (host + leader) pairs vs (host alone) at equal points. Exercises the leader’s aura on a real bodyguard.
- **-aura-uplift** — for SUPPORT characters whose value lives entirely in buffing. Measures the win-rate delta between (client + support) vs (client alone) and converts to points via a fixed linear factor.

Strengths. The bisection sees terrain, charge variance, CP, and objective contest. Coupling between rules is preserved.

Weaknesses. A unit that beats average opponents but loses catastrophically to one matchup is priced *too cheaply* by the bisection (the bad matchup is averaged away in the win-rate). It also takes 1–10 minutes per unit and ~24 hours for the full catalogue.

13.2 Track 2 — Equilibrium (closed-form log-LSQ)

`code/equilibrium.py`; output `data/equilibrium_points.json` and `data/equilibrium_points_phase2.json`

The premise: unit value is a function over the space of enemies it faces. The honest answer to “what should this unit cost?” is the solution of a symmetric zero-sum game over the catalogue.

For every pair (i, j) of units, compute the analytic per-model damage $D[i, j]$ inflicted by one i on one j per turn (hit / wound / save / FNP / Lethal / Sustained / Twin-Linked / Devastating / Anti-X). Then the time-to-kill matrix

$$T[i, j] = \frac{\text{wounds}(j)}{D[i, j]}$$

and the log-advantage matrix

$$R[i, j] = \frac{1}{2} \log \left(\frac{T[j, i]}{T[i, j]} \right).$$

The factor $\frac{1}{2}$ and the direction $T[j, i]/T[i, j]$ come from the mutual-destruction equilibrium: fielding p_j worth of i vs p_j worth of j , the side that kills first should be the cheaper-per-model unit. Solving $(p_i/p_j)^2 = T[j, i]/T[i, j]$ gives R above.

The fair-trade condition $\log p_i - \log p_j \approx R[i, j]$ is then solved in closed form by the row mean over valid entries:

$$\log p_i = \frac{1}{|V_i|} \sum_{j \in V_i} R[i, j] + \text{anchor shift},$$

where V_i is the set of j such that both $T[i, j]$ and $T[j, i]$ are finite. One unit (Intercessor Squad at 16 pts/model) is pinned to fix the overall scale.

Phase ladder.

- ✓ **Phase 1** — shooting only, with the keyword suite above.
- ✓ **Phase 2** — shoot + melee. Per-attacker role-weighted mix (SHOOTY 0.90, HEAVY 0.80, DUAL 0.55, HORDE 0.50, MELEE 0.20, SUPPORT 0.50).
- ✓ **Phase 3** — defensive integration audit. `compute_phase3_audit()` flags 69 high-FNP / 21 invuln-cliff / 819 multi-wound units; surfaced BSData mapper FNP/invuln drop bugs (subsequently fixed in tasks A12 / A21).
- ✓ **Phase 4** — tactical-utility term `tactical_value(u)` multiplicative overlay on Phase 2. Anchor-relative log-shift keeps Intercessor pinned; weights grid-search calibrated on 9 anchors (RSS 38932 $\rightarrow$ 33666). This is **Goal D**.
- ✓ **Phase 5** — meta-weighting using May 2026 Warp Friends faction shares. Re-weighted LSQ; high-meta-share factions price up monotonically.
- ✓ **Phase 6** — Nash mixed-strategy solve on the antisymmetric trade-ratio payoff matrix via `scipy.optimize.linprog`. Pearson $r = 0.968$ vs Phase 5 (Nash correctly collapses on Warlord Titan as universal dominant unit; pricing-wise extreme but mathematically right).
- ✓ **Phase sim-driven** — replaces the closed-form pairwise damage matrix with one MEASURED from the simulator. Per ordered pair (i, j) of a curated diagnostic set, run n full `Battle()` runs at equal points, take the observed win rate $r[i, j]$, and feed $R[i, j] = \text{logit}(r[i, j])$

into the same row-mean closed form. This is the first equilibrium phase that actually sees faction army rules, detachment passives, leader auras, movement, charges, and OC contests — everything Jake’s simulator work added since the closed-form phases were written. Snapshot at `data/equilibrium_points_simdriven.json`; regenerate with `python -m code.equilibrium_simdriven`. The default diagnostic set is ~ 28 anchors spanning the role $\times$ faction grid (overnight-tractable); `-full` measures every shooting-fittable unit (multi-day cron territory). Units outside the measured set inherit their Phase 1 closed-form value verbatim and are tagged `phase1_fallback` in the snapshot.

Strengths. Fast (analytic, no simulation). Exposes the pairwise structure — best/worst matchups for every unit are directly readable off R .

Weaknesses. Distance-conditional keywords (Rapid Fire, Melta, Heavy, Lance) and battlefield context (terrain, charge variance, objective pressure) are deferred to later phases. Phase 4 is where the analytic and empirical tracks finally have to agree.

13.3 Why two tracks

The bisection captures emergent dynamics the analytic solver can’t see. The equilibrium captures pairwise rock-paper-scissors structure the bisection averages away. The *divergence* between the two prices for the same unit is more informative than either price alone:

- $\text{Balancer} \approx \text{Equilibrium} \implies$ both methods agree; high confidence in the price.
- $\text{Balancer} \ll \text{Equilibrium} \implies$ unit has good pairwise stats but loses to battlefield context (slow speed, no objective play, hard counter in the sim’s terrain).
- $\text{Balancer} \gg \text{Equilibrium} \implies$ unit gets value from non-damaging utility the analytic model doesn’t see yet (this is the Goal D / Phase 4 signal).

14 Rule-citation system

Every 10e rule the simulator implements — detachment flag, leader ability, stratagem effect, simulator-side gate (Mob Rule, WAAAGH!, Shadow in the Warp, etc.) — carries a verbatim Wahapedia citation. This is enforced by CLAUDE.md §10 and the `scripts/audit_rules.py` script.

14.1 Why

The simulator’s behaviour must be auditable against canonical rules text. Without citations the model drifts: a flag named `bonus_to_hit_when_led` can quietly start to mean “+1 to hit” instead of the correct “re-roll Hit rolls of 1”, and the next calibration sweep reports false MAE improvements driven by an invented rule. This actually happened during the May 2026 Awakened Dynasty work, which is the failure mode the policy exists to prevent.

14.2 Format

Each citation lives in `data/rule_citations.d/*.json` (sharded by category: detachments, leaders, stratagems, judgement_tokens, keywords_and_mechanics) or in the top-level `data/rule_citations.json`. Required fields per entry:

- `source` — Wahapedia URL, or, if not on Wahapedia, the canonical free reference (Goonhammer datasheet readout, GW errata PDF).
- `rule_name` — exact in-game name (e.g. “Reanimation Protocols”, “Mob Rule”).
- `quoted_text` — verbatim rules text, including the “While ...” trigger half.

- **trigger** — when the rule fires, in plain English.
- **effect** — what changes, including “(approximation)” if the simulator’s behaviour diverges from canonical.
- **scope** — one of **army-wide**, **unit-led**, **character-only**, **phase-gated**, **keyword-gated**, **weapon-keyword**.

Key format: `<REGISTRY_NAME>.<field>` for detachment / leader / stratagem flags, or a dotted path for simulator gates (e.g. `simulator.big_guns_never_tire`).

14.3 Enforcement

`scripts/audit_rules.py` walks every `Detachment`, `LeaderAbility`, registered `Stratagem`, and the hand-maintained `SIMULATOR_RULE_KEYS` list. For each rule-bearing flag set away from its default, it confirms a citation exists with all six required fields. Exit code 1 if any citation is missing, incomplete, or stale.

Run before committing any change to `code/detachments.py`, `code/leaders.py`, `code/stratagems.py`, or rule-bearing parts of `code/simulator.py` / `code/units.py`:

```
PYTHONIOENCODING=utf-8 python -m scripts.audit_rules
```

Current coverage: 95/95 active rules cited. If you can’t find a canonical citation for a rule, **stop and ask** rather than approximate.

15 Matchup-matrix calibration (`evaluate_vs_meta`)

The headline calibration metric for Goal A is the mean absolute error (MAE) between the simulator’s per-faction win rate and the real tournament aggregate. Lives in `scripts/evaluate_vs_meta.py`.

15.1 How it runs

For each ordered pair (a, b) of factions in the 10-faction list, build N random 1000-pt armies per faction and run a battle on `DEFAULT_MAP`. Average A -wins to get $w_{\text{sim}}(a, b)$. Per-faction sim win rate is the mean of $w_{\text{sim}}(a, \cdot)$ over all opponents.

15.2 Reference data

The tournament-side numbers come from the Warp Friends weekly aggregate of $\sim 10k$ games in May 2026:

Faction	Real WR
Necrons	53.2%
T’au Empire	54.5%
Thousand Sons	54.6%
Aeldari	44.4%
Orks	44.9%
Leagues of Votann	46.0% (approx)
Adeptus Astartes	48.0% (approx)
Tyrannids	48.0% (approx)
Death Guard	48.0% (approx)
Adeptus Custodes	48.0% (approx)

Factions tagged “approx” use the meta midpoint of 48% rather than an authoritative aggregate, and are flagged in the report so they don’t dominate the MAE.

15.3 Current state

MAE 5.48 pts at $N=30$ / 7.01 pts at $N=200$, target ≤ 2.0 pts. The May 2026 sprint shipped 9 of 10 faction army rules (Tyranids, Drukhari, AdMech, GSC, World Eaters, Death Guard FNP + Contagions, Thousand Sons All Is Dust, plus Necrons / Orks foundation) and 5 core 10e mechanics (Look Out Sir, Lone Operative, Deadly Demise X, Fall Back, Desperate Escape). Marines Oath of Moment + Combat Doctrines remain **[NEEDS RETRY]** — three attempts have regressed because the buffs add ~ 5 pts to Marines without concurrent re-pricing.

The $N=30$ reading carries ~ 3 pt sample noise. $N=200$ is the honest baseline. Persistent residual outliers at $N=200$: Necrons -15.2 , Aeldari $+13.9$, Thousand Sons -11.6 , T'au $+10.3$.

Blocker. The Lanchester-derived `points_cost` property on `UnitProfile` is being used as a per-model GW cost by the army builder. Until the points-per-model import fix flagged in the Phase 2 follow-ups lands (Ed's TODO), every Sweg-side calibration runs in wrong-currency. The equilibrium pipeline already anchors on `points_per_squad` so its outputs are honest.

Determinism: `PYTHONHASHSEED=0` is re-executed at startup, and the per-battle RNG is seeded by faction-index pair plus battle counter, so the same code base produces the same matrix across re-runs — otherwise dice noise alone moves MAE by ~ 3 pts between back-to-back invocations.

16 Collaboration workflow

16.1 Ownership at a glance

Area	Lead	Status
Simulation core (phases, math)	[E] Eddie	in progress
Datasheet ingestion + catalogue	[J] Jake	v1 complete
Terrain & maps	[?] TBD	not started
Visualisation (dual-mode)	[?] TBD	not started
Streamlit front end	[E] Eddie	v1 done

16.2 Picking the project back up

1. `git pull`.
2. Open this PDF and scan the unchecked items in your area.
3. Pick the smallest task that unblocks something else.
4. Commit with a clear message; tick the box in this file in the same commit.

16.3 When something is safe to merge to main

- `python -m unittest discover tests` must pass (currently 73 tests, growing).
- `python run.py -cli` must exit 0 (demo plus 100-battle calibration complete cleanly).
- **Mirror Intercessor matchup VP totals:** across 200 battles at 1000 pts on `combat_patrol`, neither side's average VP exceeds the other by more than $\sim 25\%$. This replaces the old win-rate-spread gate because the new combat model (5-round + objectives + abilities) produces more draws and decisive blowouts than the old attrition-only sim, so raw win-rate is no longer a sensitive symmetry probe.
- **No new units silently disabled:** run `python -m code.bsdata.audit` and confirm the skip-reason histogram is unchanged from the previous commit (or any new skips are accounted-for in the commit message).

- The Streamlit app still launches and the preset battles still produce charts and a watchable replay.

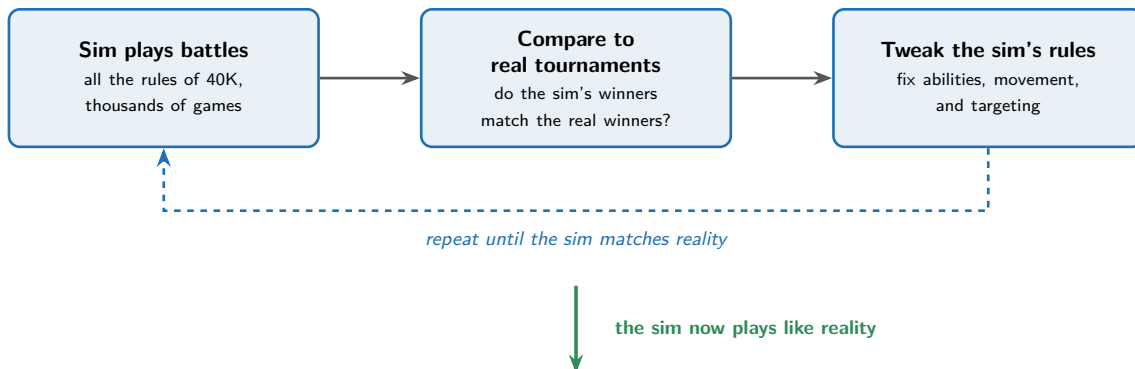
17 Glossary

Term	Meaning
AP	Armour penetration; negative integer degrading target save.
BS / WS	Ballistic / Weapon Skill; roll needed to hit.
Charge phase	Phase where units roll $2d6$ to close into base contact.
Cover	Terrain bonus to saving throws or hit penalty.
CP	Command Point; resource for stratagems and rerolls.
Lanchester score	$(H \cdot D \cdot P)^2$; theoretical effectiveness of a unit.
LoS	Line of sight; whether the attacker can “see” the target through terrain.
Stochastic mode	Simulator with real dice rolls.
Deterministic mode	Simulator using expected values, no rolls.

SwegHammer: how it works

A fairer Warhammer 40K. Step one: build a simulator that plays the game the way real tournaments do. Step two: use it to fit one points equation that prices every unit fairly from its stats.

Stage 1 – Make the simulator realistic



Stage 2 – Fit the points equation

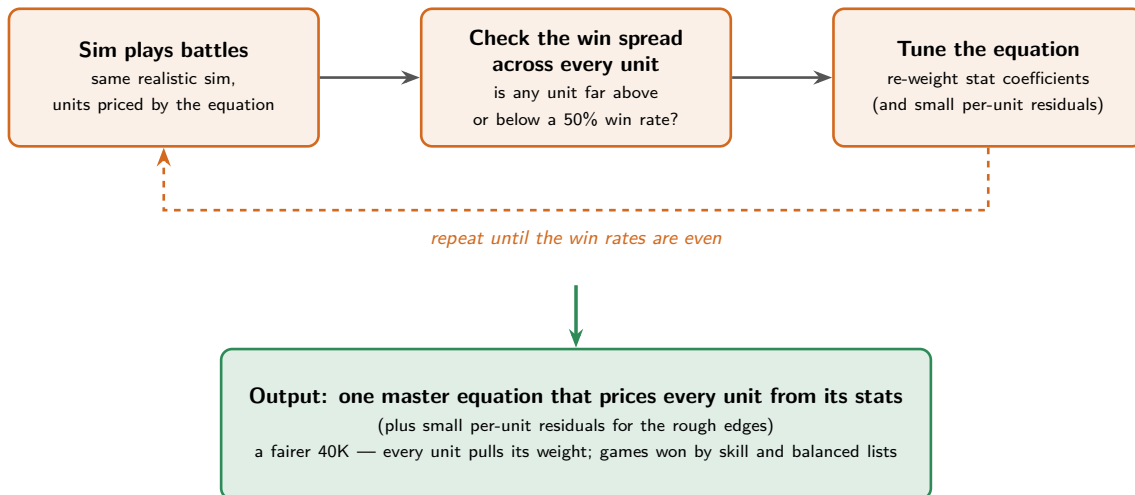


Figure 1: SwegHammer’s two-stage pipeline. Stage 1 (blue) tunes the simulator’s rules against per-faction tournament win rates. Once Stage 1 has converged, Stage 2 (orange) fits one master points equation (plus small per-unit residuals) using the same simulator to measure per-unit win rates. The two loops have different feedback signals and must not be conflated. Diagram source: `OVERVIEW.tex`, also distributed as a standalone single-page `OVERVIEW.pdf` for sharing outside the project.